

# Clase 3:

## Estructuras de Control : Decisión

### Introducción a las estructuras algorítmicas de control

Las **estructuras algorítmicas de control** son construcciones fundamentales en programación que permiten modificar el flujo de ejecución de un programa.

Por defecto, un programa ejecuta sus instrucciones de manera secuencial (de arriba hacia abajo). Sin embargo, muchos problemas requieren que ciertas instrucciones se ejecuten solo bajo determinadas condiciones.

Las estructuras de control permiten:

- Tomar decisiones.
- Ejecutar bloques alternativos.
- Controlar el comportamiento dinámico del programa.

Dentro de ellas se encuentran las estructuras de decisión, repetición y selección múltiple. En esta unidad se estudian las estructuras de decisión.

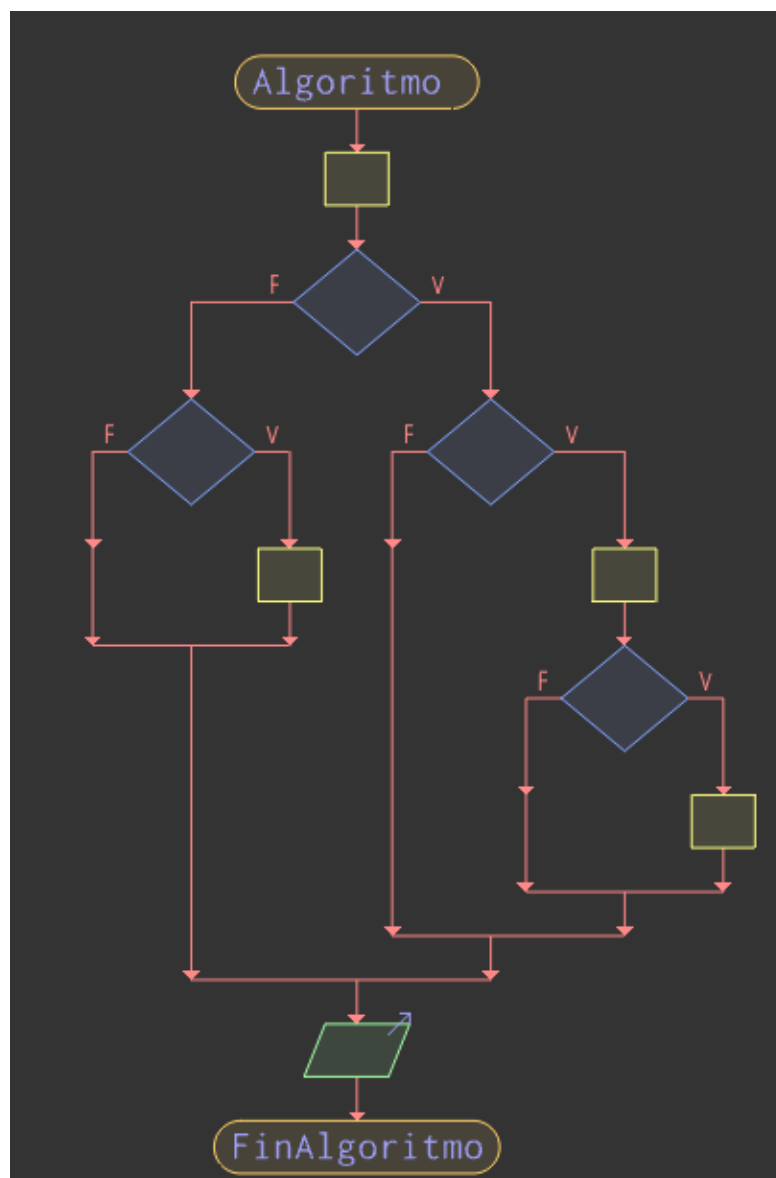
# Estructura de decisión

Una estructura de decisión es un mecanismo que permite **evaluar una condición lógica** y, en función del resultado (verdadero o falso), **determinar qué bloque de instrucciones debe ejecutarse**.

Estas estructuras se basan en expresiones booleanas, es decir, expresiones que devuelven un valor lógico:

- Verdadero (true)
- Falso (false)

Su objetivo es introducir bifurcaciones en el flujo del programa, permitiendo que este responda de manera diferente según los datos de entrada o el estado interno del sistema.



# Operadores relacionales

Los operadores relacionales permiten **comparar dos valores** y determinar la relación existente entre ellos.

El resultado de una operación relacional es siempre un valor booleano.

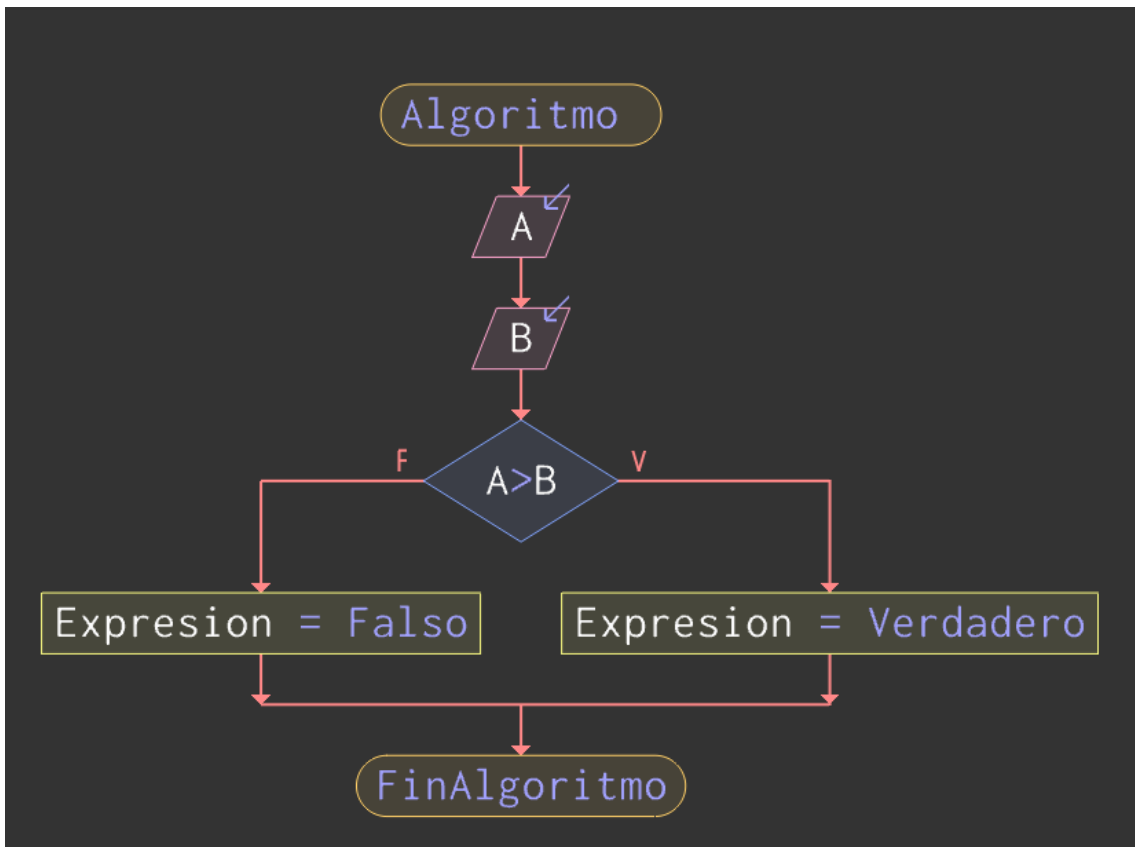
Principales operadores:

- > mayor que
- < menor que
- >= mayor o igual que
- <= menor o igual que
- == igual a
- != distinto de

Estos operadores son fundamentales para construir condiciones en estructuras de decisión, ya que permiten evaluar relaciones numéricas o de igualdad entre variables.

Ejemplo:

Una expresión como  $a > b$  será verdadera si el valor de  $a$  es mayor que el de  $b$ , y falsa en caso contrario.



# Estructura If

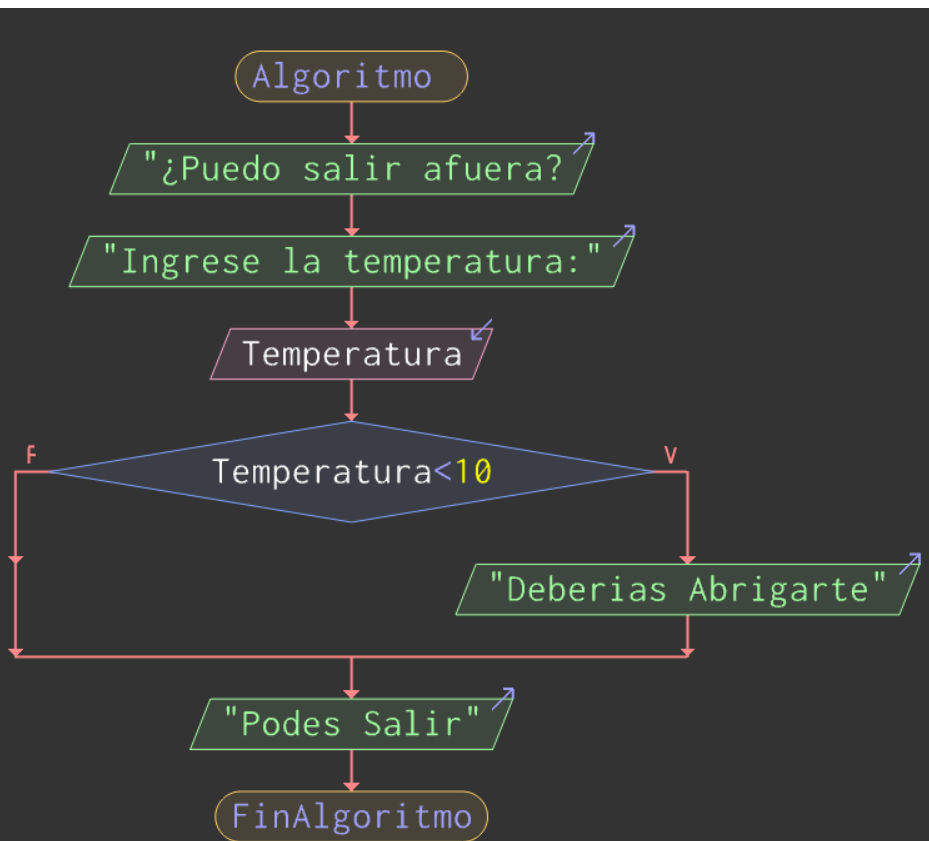
La estructura **if** (si) permite **ejecutar un bloque de instrucciones únicamente cuando una condición es verdadera**.

Su funcionamiento consiste en:

1. Evaluar una expresión booleana.
2. Ejecutar el bloque asociado si el resultado es verdadero.
3. Continuar con la ejecución normal del programa si es falso.

Representa la forma más básica de decisión condicional y permite introducir bifurcaciones simples en el flujo de control.

**Por ejemplo:** Un programa que nos diga si podemos salir afuera.



1) El programa **imprime por pantalla** un mensaje pidiéndole al usuario que ingrese la temperatura.

2) El usuario **ingresa por teclado** la temperatura.

3) El programa **evalúa una condición lógica**, en este caso: ¿Es Temperatura menor que 10°C?

Si la condición es **Verdadera** se ejecutara un bloque que nos diga "Deberías Abridarte".

De lo contrario, si la condición es **Falsa**, el bloque continua su ejecución

4) Se **imprime por pantalla** nuestro mensaje correspondiente.

# Operadores lógicos

Los operadores lógicos permiten **combinar múltiples expresiones booleanas** para formar condiciones más complejas.

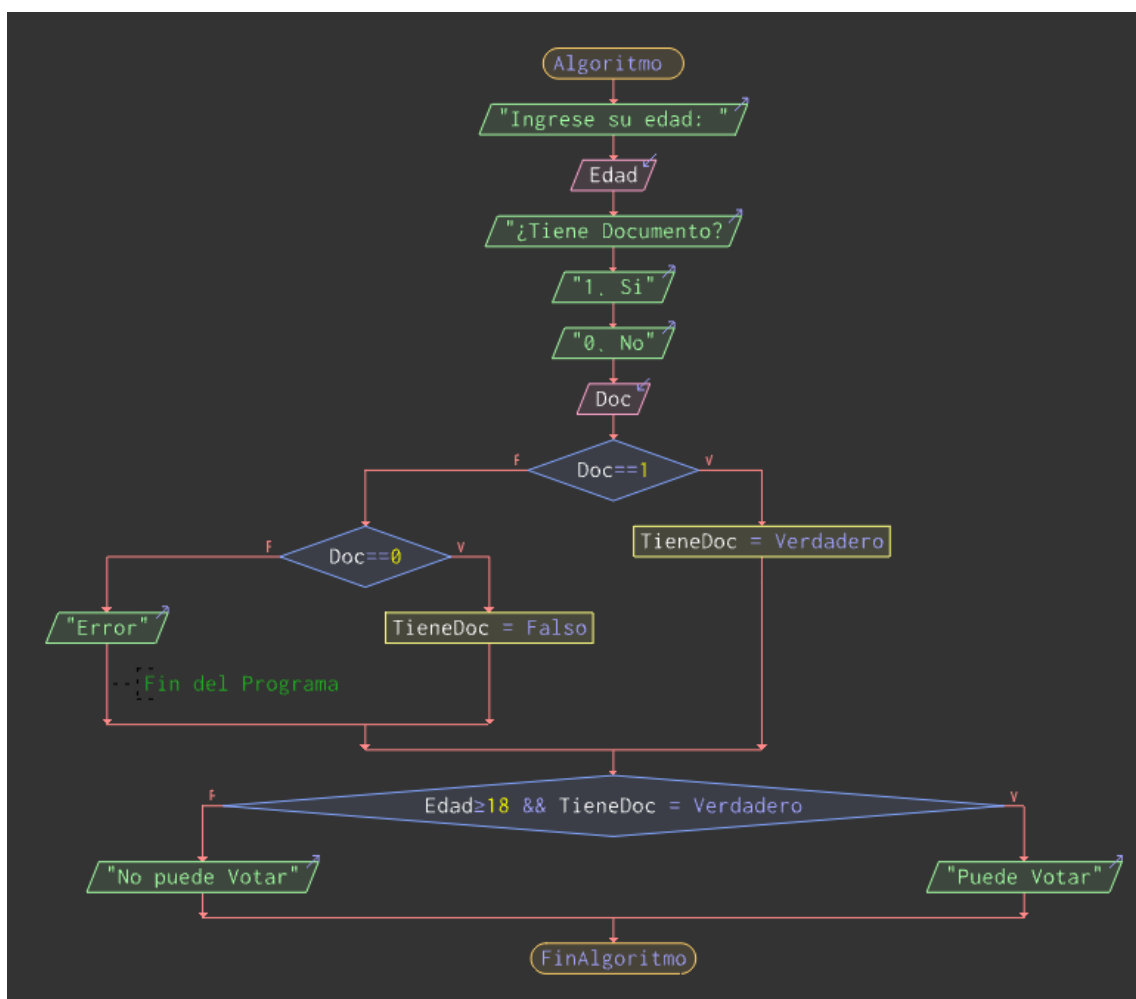
Operadores principales:

- && (AND): la expresión es verdadera solo si todas las condiciones son verdaderas.
- || (OR): la expresión es verdadera si al menos una condición es verdadera.
- != (NOT): invierte el valor lógico de una expresión.

Estos operadores permiten modelar situaciones donde intervienen múltiples criterios de decisión.

**Por ejemplo:** Un programa que nos diga si podemos votar.

Una condición como (edad >= 18 && tieneDocumento == true) solo será verdadera si ambas condiciones se cumplen simultáneamente.



## Estructura If-Else

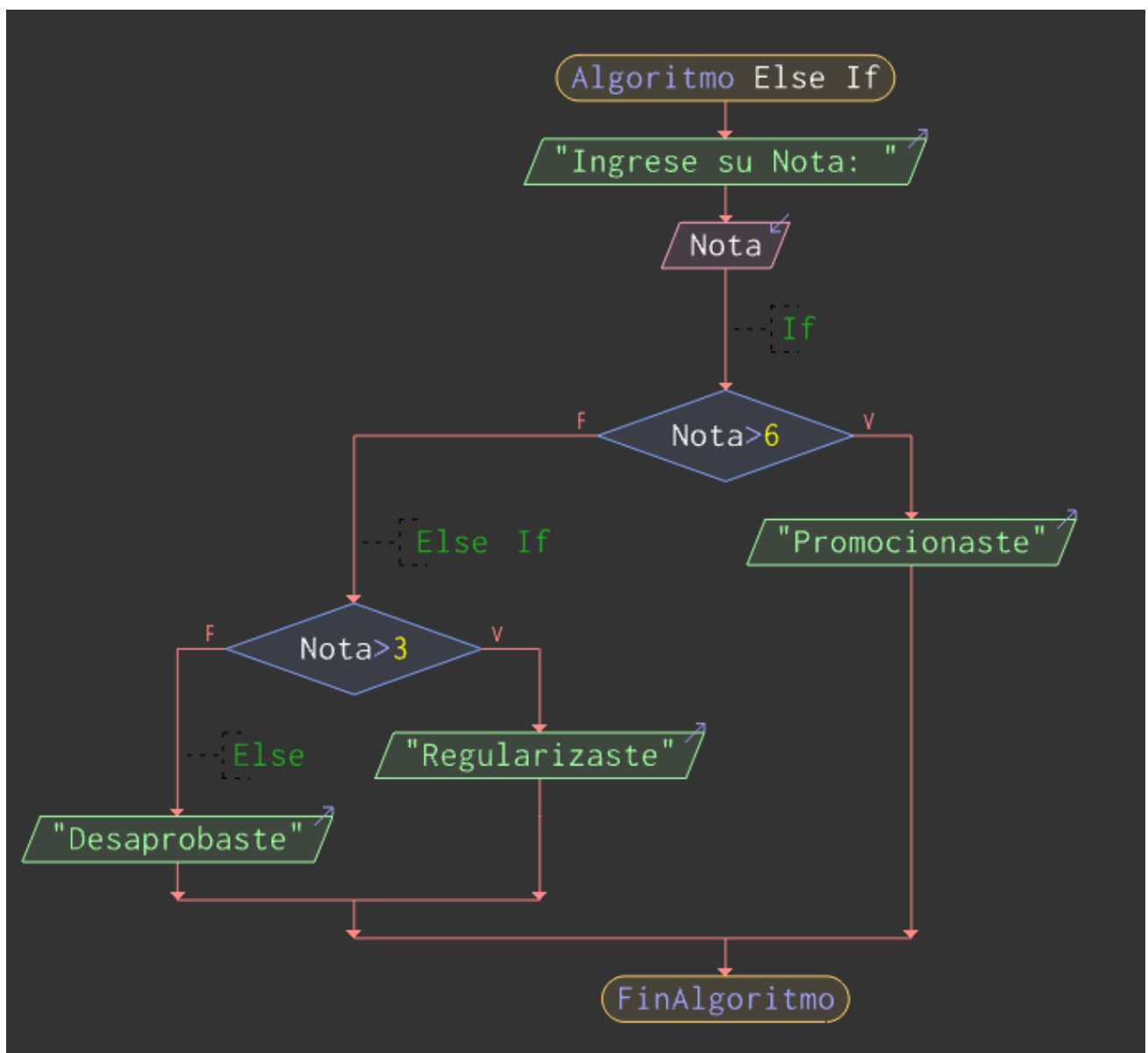
La estructura **if-else** amplía la funcionalidad del **if**, permitiendo definir un bloque alternativo de ejecución cuando la condición evaluada resulta falsa.

Su comportamiento es excluyente:

- Si la condición es **verdadera**, se ejecuta el bloque del **if**.
- Si la condición es **falsa**, se ejecuta el bloque del **else**.

Solo uno de los bloques se ejecuta en cada evaluación.

También puede extenderse mediante **else if**, permitiendo evaluar múltiples condiciones en secuencia, generando una cadena de decisiones.



Si quisiéramos programar este mismo algoritmo en C++, la estructura sería la siguiente:

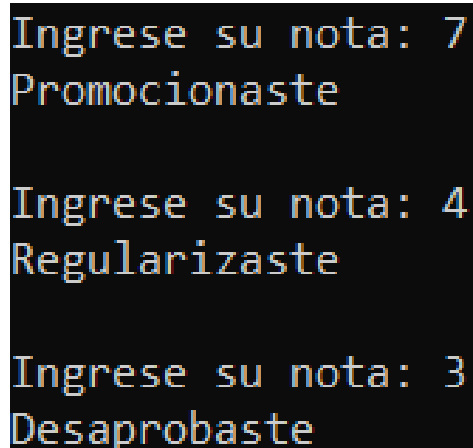
```
#include <iostream>

using namespace std;

int main() {
    int Nota;

    cout<<"Ingrese su nota: ";
    cin>>Nota;
    if(Nota>6){
        cout<<"Promocionaste"<<endl;
    }
    else if(Nota>3){
        cout<<"Regularizaste"<<endl;
    }
    else{
        cout<<"Desaprobaste"<<endl;
    }
    cout<<endl;
    return 0;
}
```

Dando como resultado:



```
Ingrese su nota: 7
Promocionaste

Ingrese su nota: 4
Regularizaste

Ingrese su nota: 3
Desaprobaste
```

# Estructura Switch

La estructura switch es una estructura de selección múltiple que permite **evaluar el valor de una variable y compararlo contra distintos casos posibles**.

Su funcionamiento consiste en:

1. Evaluar una expresión (generalmente una variable).
2. Comparar su valor con distintos casos definidos.
3. Ejecutar el bloque correspondiente al caso coincidente.

Se utiliza principalmente cuando se necesita evaluar múltiples valores específicos de una misma variable, especialmente en menús o selección de opciones.

A diferencia de if, el switch se emplea para comparaciones directas de igualdad y no para evaluar rangos o condiciones complejas.

Por ejemplo: Un programa que nos diga cuantos helados de “x” sabor compró un cliente

